Jarvis-Style AI Screen Project

Conversation summary & locked-in project decisions

1. Project Origin

The project started from a set of salvaged HP laptop parts (approx. 2010-2011 era, motherboard marked 605747-001): Intel CPU motherboard, 320GB Seagate HDD, 2x SO-DIMM RAM (2GB + 2GB), CPU heatsink + fan, DVD drive, 6-cell battery, LCD screen with matching LVDS cable, webcam module, and a card reader / USB daughterboard. Together these form nearly a complete laptop's internals, minus the chassis, keyboard, and trackpad.

2. Project Goal

Build an AI-integrated screen inspired by (not a literal replica of) Jarvis from the Iron Man films: a device that can be asked questions by text (and eventually voice) and responds with useful, explained answers, visuals, and data.

3. Physical Build Decisions

- Open-frame build - no enclosure or case. Components are intentionally exposed/visible rather than hidden.
- All physical positioning, mounting, and assembly will be handled by the user directly (not part of the software planning).
- The screen connects to the motherboard via its original matching LVDS/eDP cable - no separate controller board needed.

4. Software Architecture - Final Scope

The hardware (2010-era mobile CPU, 4GB RAM, no dedicated GPU) cannot run a capable AI model locally. The agreed approach: the hardware acts as the 'body' (input, display, voice output), while the AI 'brain' runs via a cloud LLM API call over Wi-Fi.

Core feature decisions:

Feature	Final decision
Input method	Text input first. Voice input added later, once the core system works reliably.
Output method	Text on screen + spoken voice output (offline text-to-speech) from the start.
General Q&A;	Routed to a cloud LLM (Claude API).
Time	Read directly from the system clock - no external service needed.

Weather	Open-Meteo API - free, no signup, no API key required. Chosen over Google's Weather API, which requires a Google Cloud billing account even for free-tier use. Must display current temperature, humidity, and wind speed (plus other available conditions).
Math / Probability / Statistics	Textbook-style calculations only (e.g. probability distributions, standard deviation, etc.), computed exactly using local Python (scipy/numpy), with the LLM explaining the result in plain language. Real-world/internet-pulled probability (e.g. sports match win probability) was considered and explicitly dropped from scope for now, to keep the first working version simpler.
Graphing / visual analysis	Generated locally using matplotlib, displayed on screen.
On-screen UI	To be designed from reference images the user will provide for each situation/screen - not yet finalized.

5. Workflow Decision

Design/planning discussions happen in this chat. Actual code writing happens in a separate, dedicated coding chat.

6. Cost / API Usage Reality Check

- Claude.ai (chat) and the Claude API are separate products with separate limits - the API is not free.
- The Claude API is pay-as-you-go, billed per request based on tokens used. There is a user-configurable monthly spend limit (recommended to set a low cap, e.g. around $5, as a safety net).
- At personal hobby-usage scale with the cheapest/fastest model (Haiku), realistic cost is roughly $1-5/month, not more - a handful of short Q&A; exchanges per day costs a small fraction of a cent each.
- Anthropic does not publish a free tier for the Claude API. Some other providers (e.g. Google AI Studio/Gemini, Groq) do offer genuinely free, rate-limited API tiers with no credit card required, as a possible zero-cost alternative to explore if a paid API is not wanted - with the trade-off of a different model/provider and more restrictive rate limits.

Note: this document reflects the state of the discussion as of the conversation it was generated from. The UI design, voice input, and any later additions are not yet finalized and will be refined in future discussion.